

Trabajo Práctico II - Tecnología Digital VI

Universidad Torcuato Di Tella

Petra Leoni, Candelaria Luna, Lucía Pizi

19 de Octubre de 2025

Contenidos del trabajo

1	Introducción	1
2	Análisis exploratorio de datos	1
3	Variables adicionales	2
4	Armado de conjunto de validación	4
5	Modelos predictivos y búsqueda de hiperparámetros	4
6	Atributos más significativos	4
7	Conclusiones	5

1 Introducción

Este trabajo aborda el problema de predicción de skip en canciones de Spotify aplicando técnicas de aprendizaje supervisado. El objetivo es estimar la probabilidad de que un usuario salte una canción a partir de información sobre el usuario, la pista y su contexto de reproducción.

El proceso incluyó las etapas clásicas del ciclo de machine learning: limpieza y preparación de datos, ingeniería de atributos, entrenamiento y evaluación de modelos. Se siguió un enfoque iterativo: partimos de modelos simples como árboles de decisión, incorporamos nuevas variables y combinaciones de atributos para mejorar la capacidad predictiva, y avanzamos hacia ensambles como Random Forest y XGBoost. Finalmente, se ajustó un blend entre ambos modelos para optimizar el desempeño, evaluado mediante la métrica ROC-AUC. Todo el desarrollo se realizó con validación por usuario para evitar data leakage y garantizar la robustez del sistema.

2 Análisis exploratorio de datos

Antes de entrenar los modelos, se realizó un análisis exploratorio de datos con el objetivo de comprender mejor la estructura del dataset y las relaciones entre las variables. Esta exploración sirvió de base para definir estrategias de ingeniería de atributos y orientar la selección de las variables más relevantes para el modelado posterior. A continuación se presentan dos visualizaciones que resumen algunos de los patrones más relevantes observados en los datos.

En primer lugar, se evaluaron patrones de *skip* (target = 1) según distintas variables temporales y de contexto. Uno de los hallazgos más notables fue la variación en la tasa de skips a lo largo del día: las horas de la tarde y la noche presentan mayores probabilidades de que el usuario salte una canción, mientras que la mañana y la madrugada muestran tasas más bajas. Este comportamiento motivó la creación de variables temporales como `hour`, `daytime` e `is.weekend`, y combinaciones con otras dimensiones del contexto (por ejemplo, `platform.daytime`).



Figure 1: Tasa de *skip* a lo largo del día. Las franjas de mayor actividad (tarde/noche) presentan una mayor tasa de saltos.

También se observó una diferencia clara en la tasa de skips según el tipo de contenido. El contenido musical mostró una probabilidad de skip considerablemente mayor que los podcasts, lo cual refleja un patrón de consumo más volátil en canciones individuales y mayor estabilidad en contenido narrativo. Este análisis justificó la inclusión de variables categóricas como `content_type` y el uso de *bin counting* (mean target encoding) para capturar promedios de skip por artista, álbum o categoría de contenido.

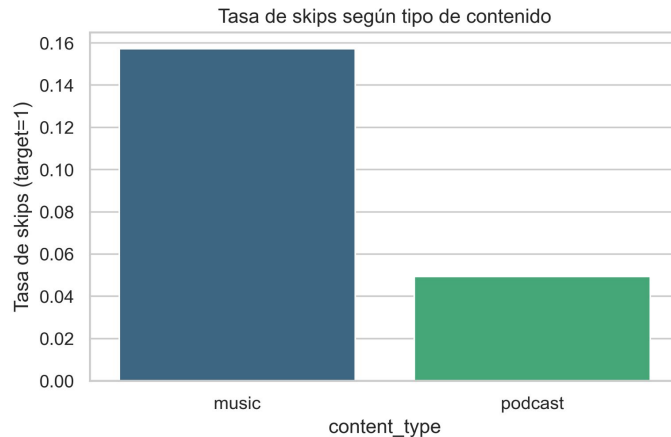


Figure 2: Tasa de *skip* según tipo de contenido. Las canciones presentan una mayor proporción de skips que los podcasts.

En síntesis, el análisis exploratorio permitió detectar patrones asociados al momento del día, al tipo de contenido y al comportamiento del usuario, lo que orientó el diseño de las variables adicionales que luego demostraron mayor relevancia en la performance del modelo.

3 Variables adicionales

Durante el desarrollo del trabajo, el foco principal estuvo puesto en la ingeniería de atributos, dado que las mayores mejoras de performance provinieron de incorporar nueva información relevante más que del ajuste de hiperparámetros.

En una primera etapa, se evaluó la importancia de las variables originales del dataset provisto, con el objetivo de identificar qué aspectos del usuario, del contenido y del contexto de reproducción aportaban mayor capacidad predictiva. A partir de ese análisis inicial, se diseñaron nuevas variables derivadas de las columnas existentes, buscando capturar patrones de comportamiento y hábitos de escucha.

En una segunda etapa, se incorporaron variables de agregación y contexto, como el promedio de orden de reproducción por usuario, la cantidad de artistas y álbumes escuchados, indicadores temporales (hora, día, fin de semana) y tipo de contenido. También se construyeron representaciones categóricas más interpretables, como la región geográfica a partir del país de conexión y el modo de reproducción (shuffle, offline, incógnito).

Posteriormente, dado que el tuning de hiperparámetros no producía mejoras significativas, se decidió concentrar los esfuerzos en seguir expandiendo el conjunto de atributos. Se incorporaron técnicas de bin counting (mean target encoding con K-Fold sin leakage) sobre variables categóricas de alta cardinalidad, como artista, álbum, país, plataforma y tipo de contenido, lo cual permitió capturar relaciones promedio entre categorías y la variable objetivo sin introducir fuga de información.

En la última etapa se agregaron variables de historial y comportamiento temporal, que resultaron determinantes para alcanzar los mejores resultados del modelo final. Entre ellas, la racha de skips consecutivos (streaks). Esta incorporación y algunas más fueron las que llevaron el AUC público de Kaggle hasta 0.802, el mejor obtenido por el equipo.

Durante todo el proceso, se analizó la importancia de las variables mediante los gráficos de feature importance del modelo XGBoost, observando cómo las nuevas variables desplazaban a las originales entre las más relevantes.

Listado de variables creadas y su origen

A continuación se enumeran las variables nuevas generadas durante el proceso de preprocesamiento e ingeniería de atributos, junto con la columna o combinación de columnas a partir de las cuales fueron creadas:

- **target** e **is_test**: variables binarias que indican respectivamente si la reproducción terminó por skip y si pertenece al conjunto de test.
Fuente: `reason_end` (valor "fwdbtn" o nulo).
- **user_order** y **user_order_mean**: orden secuencial de reproducciones dentro de cada usuario y su promedio.
Fuente: `ts` (orden temporal de reproducciones por `username`).
- **Variables temporales:** `hour`, `weekday`, `month`, `is_weekend`, `daytime`, `hour_sin`, `hour_cos`.
Fuente: `ts` (timestamp). Incluyen hora, día, mes, indicador de fin de semana, franja horaria categórica y codificación cíclica de la hora.
- **Variables de historial de usuario:** `prev_target`, `mean_last3`. Capturan el comportamiento reciente del usuario y su tendencia a realizar skips en las últimas reproducciones.
Fuente: `target` (historial previo por usuario con desplazamiento temporal y ventanas móviles).
- **dt_prev_min**: minutos transcurridos desde la reproducción anterior del mismo usuario.
Fuente: diferencias entre timestamps `ts` dentro de cada usuario.
- **user_activity_log**: logaritmo del número total de reproducciones por usuario.
Fuente: conteo de filas por `username`.
- **Variables de contenido:** `content_type`, `artist_count`, `album_count`, `content_popularity`. Describen el tipo y la popularidad del contenido, así como la frecuencia de aparición de artistas y álbumes.
Fuente: `master_metadata_album_artist_name`, `master_metadata_album_album_name`, `spotify_track_uri`, `spotify_episode_uri`, `audiobook_uri`.
- **region**: región geográfica agrupada según país de conexión.
Fuente: `conn_country` (mapeo país-región).
- **shuffle_ratio**: proporción de reproducciones en modo shuffle por usuario.
Fuente: `shuffle`.
- **playback_mode**: modo combinado de reproducción (shuffle/offline/incógnito).
Fuente: combinación de `shuffle`, `offline` e `incognito_mode`.
- **same_artist_prev**, **same_track_prev**, **mins_since_artist**: indicadores de repetición o cercanía temporal con el mismo artista o canción.
Fuente: `ts`, `spotify_track_uri`, `master_metadata_album_artist_name`.

- **streak_skip_prev_log** y **streak_balance**: longitud y balance de rachas consecutivas de skips y plays, suavizadas logarítmicamente.
Fuente: **target** (historial previo por usuario).
- **Variable cruzada platform_daytime**,
Fuente: combinación entre **platform** y **daytime**.
- **Bin countings** (`<col>.bin`): codificaciones numéricas obtenidas mediante *mean target encoding* (K-Fold sin fuga) sobre variables categóricas de alta cardinalidad como **platform**, **conn_country**, **content_type**, **daytime**, **region**, **artist**, **album**, **audiobook** y **playback_mode**.

Estas variables fueron creadas a partir de la información temporal, de comportamiento y de contenido del dataset original. Su incorporación permitió capturar patrones más complejos en el uso de la plataforma y contribuyó significativamente a la mejora del AUC del modelo.

4 Armado de conjunto de validación

El conjunto de validación se construyó dividiendo las observaciones por usuario, de manera tal que cada usuario aparezca únicamente en uno de los conjuntos (entrenamiento o validación). Esta estrategia permite evitar data leakage, ya que las reproducciones de un mismo usuario suelen estar altamente correlacionadas: si aparecieran en ambos subconjuntos, el modelo podría “aprender” patrones individuales y sobreestimar su desempeño.

Además, este esquema refleja un escenario más realista del problema, en el cual el sistema debe predecir el comportamiento de usuarios no vistos durante el entrenamiento. De esta forma, la validación mide la verdadera capacidad de generalización del modelo y asegura que las métricas obtenidas sean representativas de su rendimiento esperado en nuevos datos.

5 Modelos predictivos y búsqueda de hiperparámetros

A lo largo del trabajo se avanzó de manera progresiva en la complejidad de los modelos utilizados. En una primera etapa se entrenó un árbol de decisión simple, con el objetivo de familiarizarnos con el problema y obtener un punto de partida para comparar futuros modelos.

Posteriormente se implementó un Random Forest, aprovechando las ventajas del ensamble de múltiples árboles para reducir la varianza y mejorar la capacidad de generalización. Se realizaron pruebas de tuning sobre distintos hiperparámetros; como la cantidad de árboles, la profundidad máxima, entre otras. Sin embargo, no se observaron mejoras sustanciales respecto al modelo con parámetros por defecto. De hecho, los mejores resultados de AUC en Kaggle se mantuvieron utilizando la configuración básica, lo cual sugiere que el modelo ya estaba bien calibrado para el tamaño y tipo de datos del problema.

A continuación, se probó un modelo de XGBoost, un método de boosting que entrena árboles de forma secuencial. También se llevaron a cabo pruebas de regularización y ajuste de hiperparámetros, pero las mejoras obtenidas fueron marginales. Por este motivo, se optó por mantener la configuración básica y priorizar el avance sobre otras etapas del proyecto, como ingeniería de atributos y variables adicionales. Además, al no llevar a cabo los procesos de tuning que demandaban tiempos prolongados de ejecución, pudimos iterar más rápido y evaluar con mayor agilidad los efectos del feature engineering, viendo los resultados locales de AUC casi en tiempo real.

Finalmente, se incorporó un blend entre Random Forest y XGBoost, combinando sus predicciones mediante una media ponderada. Esta estrategia permitió aprovechar las fortalezas de ambos modelos: la estabilidad del Random Forest y la capacidad de ajuste fino del XGBoost. Inicialmente se probó con un peso fijo de combinación, observando mejoras en el AUC local y en Kaggle, especialmente con un valor de $w=0.6$, que otorgaba mayor peso a XGBoost. A partir de ese resultado, se implementó una optimización sistemática del parámetro w , buscando el valor que maximizara el AUC en el conjunto de validación. Este ajuste final, junto con un buen desarrollo de ingeniería de atributos, permitió obtener la mejor performance global del trabajo.

6 Atributos más significativos

El análisis de importancia de variables de XGBoost mostró que las variables relacionadas con el comportamiento reciente del usuario fueron las más influyentes en la predicción del skip. Entre ellas, se destacaron **streak_balance**, **prev_target**, **mean_last3** y **streak_skip_prev_log**, que reflejan la tendencia inmediata del usuario a saltar canciones y la continuidad de su comportamiento a lo largo de la sesión.

También resultaron relevantes las variables de contexto temporal, como `dt_prev_min` (minutos desde la reproducción anterior) y `platform_daytime_bin`, que capturan la frecuencia de interacción y el momento del día. Además, las características agregadas por usuario (`user_order_mean`, `user_activity_log`, `shuffle_ratio`) y los bin countings sobre artista, álbum y plataforma aportaron información adicional que mejoró la capacidad predictiva del modelo.

En conjunto, la importancia relativa de las variables confirma que el comportamiento reciente y el contexto de reproducción son factores determinantes en la probabilidad de que un usuario salte una canción.

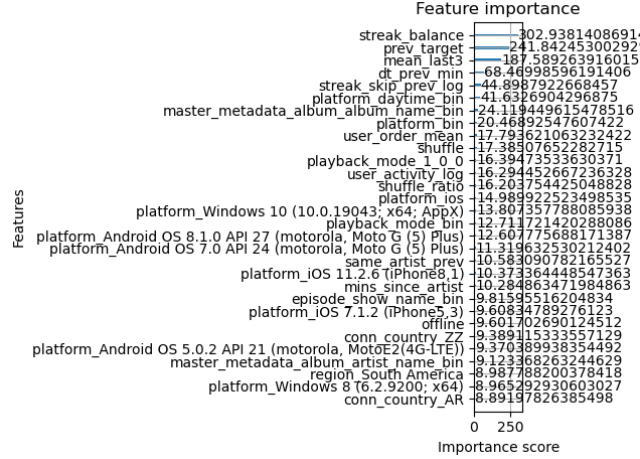


Figure 3: Importancia de atributos en la solución final

7 Conclusiones

En conclusión, el sistema desarrollado permitió abordar el problema de predicción de *skip* en Spotify de manera integral, combinando un análisis exploratorio exhaustivo, una sólida ingeniería de atributos y la aplicación de modelos de aprendizaje supervisado complementarios. Los resultados obtenidos demostraron que el desempeño del modelo no dependía únicamente del tipo de algoritmo utilizado, sino principalmente de la calidad y relevancia de las variables creadas. En particular, las variables derivadas del comportamiento reciente del usuario, del contexto temporal y del tipo de contenido fueron determinantes para mejorar la capacidad predictiva del sistema.

Durante el desarrollo se probaron distintos modelos y configuraciones, y se observó que las mayores mejoras surgían de incorporar información más representativa del comportamiento de los usuarios. Esta estrategia permitió avanzar de manera ordenada, evaluando rápidamente los efectos de cada cambio e identificando las variables más influyentes en la predicción. Asimismo, la implementación del *blend* entre Random Forest y XGBoost, con optimización del parámetro de peso w , evidenció que la combinación de modelos con diferentes sesgos y varianzas podía generar una mejora adicional en la métrica de AUC tanto a nivel local como en la evaluación pública de *Kaggle*, alcanzando un valor de 0.802 , el mejor obtenido por el equipo.

En términos metodológicos, el uso de una validación por usuario permitió garantizar la robustez de las métricas y evitar fugas de información, representando con mayor fidelidad un escenario real de predicción sobre nuevos usuarios.

En conjunto, el trabajo refleja cómo el aprendizaje automático puede aplicarse de forma efectiva a problemas reales, donde la comprensión del problema y la creatividad en la ingeniería de atributos resultan tan relevantes como la elección del modelo en sí.